EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 5

Object-Oriented Analysis — Object Modelling I

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 5 of 16

Contents

1. Week 5 Introduction

Weeks 1–4 built the foundation: Week 1 examined business needs and business software needs; Week 2 introduced object-oriented technology and principles; Week 3 introduced UML; and Week 4 explained UML's building blocks, rules, mechanisms and architecture. This week marks an important transition — **we begin the actual Object-Oriented Analysis (OOA) process**.

Students should now move from thinking *"I know what a UML class diagram is"* to thinking *"Given a real-world problem, I can identify objects/classes and represent their relationships correctly."* The course outline divides Object-Oriented Analysis (Object Modelling) into two parts: **Week 5 — Object Modelling I** (object/class, notation, links and association, generalization/specialization, CASE tool) and **Week 6 — Object Modelling II** (aggregation, inheritance, polymorphism, grouping).

2. Learning Outcomes

By the end of this week, a student should be able to:

- Explain object-oriented analysis.
- Explain the difference between an object and a class.
- Identify candidate objects/classes from a problem description.
- Distinguish relevant objects from irrelevant nouns.
- Identify attributes, operations and responsibilities of objects.
- Draw a UML class and an object diagram using correct notation.
- Explain links between objects and associations between classes.
- Identify association names, roles and multiplicity.
- Use multiplicity correctly in a class diagram.
- Explain generalization and specialization, and draw them correctly.
- Distinguish generalization from ordinary association.
- Apply object modelling to a real-world system and evaluate the result.

3. What Is Object-Oriented Analysis?

Analysis means studying a problem to understand: what exists; what users need; what activities occur; what information is involved; what rules apply; and how different parts relate to one another. In **Object-Oriented Analysis**, we examine the problem domain in terms of **objects, classes, responsibilities, relationships and behaviour**.

Consider a university that wants a student registration system. A traditional analysis might begin with *Input* □ *Process* □ *Output*. Object-oriented analysis instead asks: **who/what are the important objects?** — Student, Course, Registration, Lecturer, Programme — and then: **how are these objects related?**

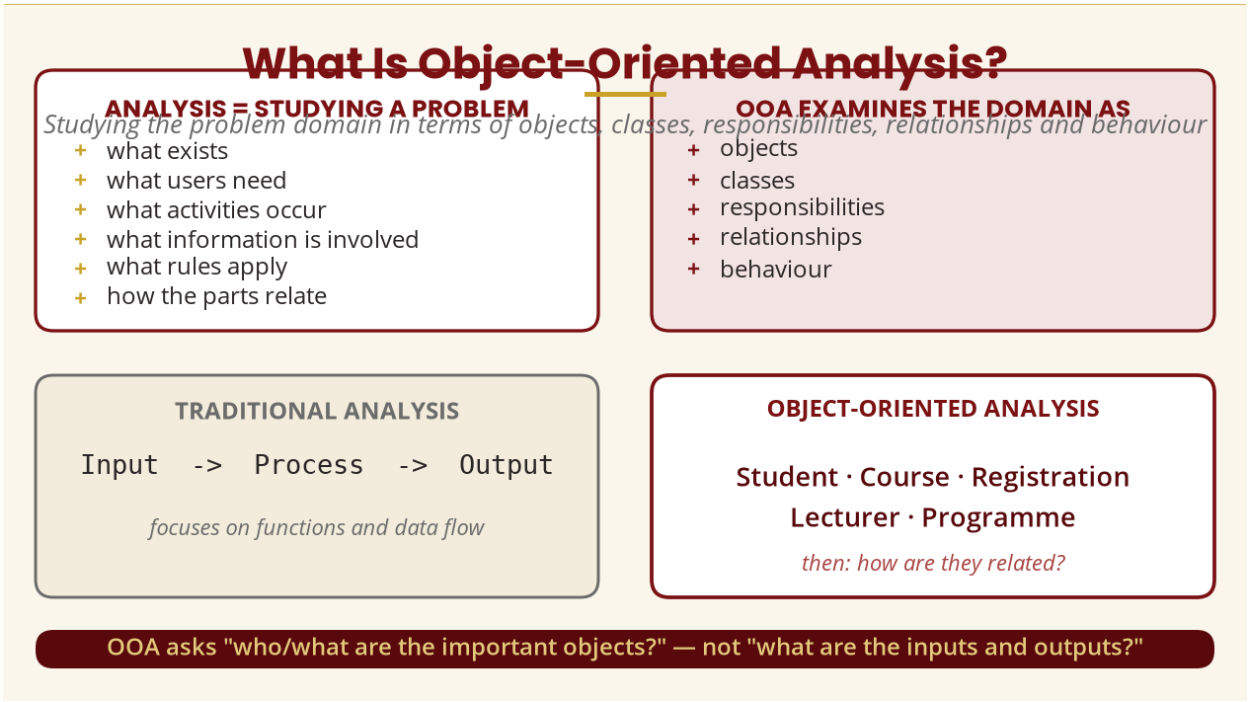

Figure 1 — Traditional analysis vs object-oriented analysis

4. The Key Question in Object Modelling

Whenever students receive a case study, teach them to ask: **"What important things exist in this problem domain?"** For a banking system — Customer, Account, Transaction, Bank, Loan, Card; for a hospital — Patient, Doctor, Nurse, Appointment, Prescription, MedicalRecord; for an online university — Student, Lecturer, Course, Assignment, Submission, Grade. These become **candidate classes**. But be careful: **not every noun in a problem description should automatically become a class.**

5. Object vs Class

A **class** is a description or specification of a group/type of objects that share common characteristics and behaviour — for example, **Student** describes what students generally have and can do. An **object** is an individual instance of a class — for example, `Francis : Student`. Here *Student* is the class and *Francis* is the object.

Think about cars: **Car** is a general concept/class, while individual cars — Toyota Corolla ABC123, Honda Civic DEF456, Mazda Demio GHI789 — are objects. The class describes common characteristics; each object has its own values. Two objects, `student001 : Student` (John Banda, BSc ICT, year 3) and `student002 : Student` (Mary Phiri, BSc ICT, year 2), both belong to the same class.

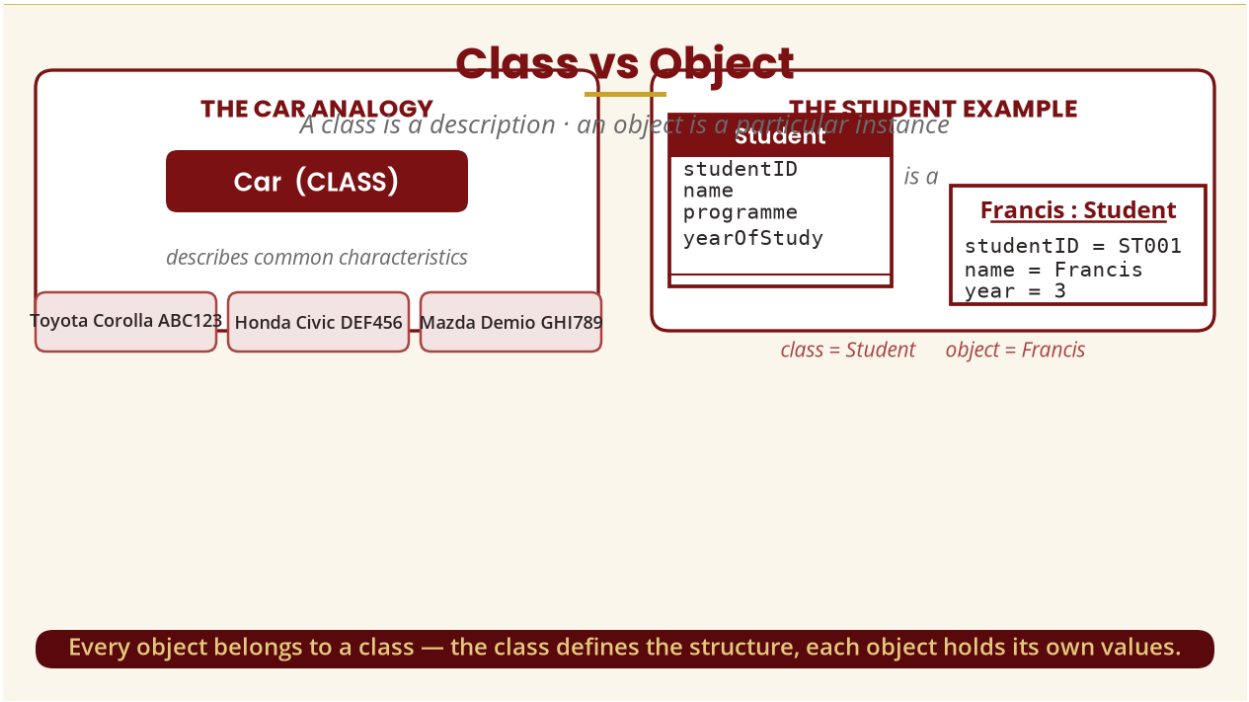

Figure 2 — Class vs object: the car and student analogies

6. UML Class Notation

The standard UML class notation is a rectangle divided into compartments. The three compartments are commonly used for: (1) **class name**, (2) **attributes**, and (3) **operations**. A good class name represents a meaningful concept in the problem domain — Student, Course, Account, Payment, Employee, Product, Order. Poor names include Thing, Data, Information, Stuff, Object1, ClassA.

An **attribute** describes a property or characteristic of an object: for Student — studentID, name, programme, yearOfStudy; for Course — courseCode, courseName, creditHours; for Account — accountNumber, accountType, balance. A common attribute notation is `visibility name : type` — e.g. `- studentID : String`, where − is the visibility, *studentID* is the name and *String* is the data type.

An **operation** represents behaviour/responsibility associated with a class: Student — registerCourse(), dropCourse(), viewResults(); BankAccount — deposit(), withdraw(), checkBalance(); Course — addStudent(), removeStudent(), assignLecturer(). Operation names should come from the requirements and responsibilities discovered during analysis.

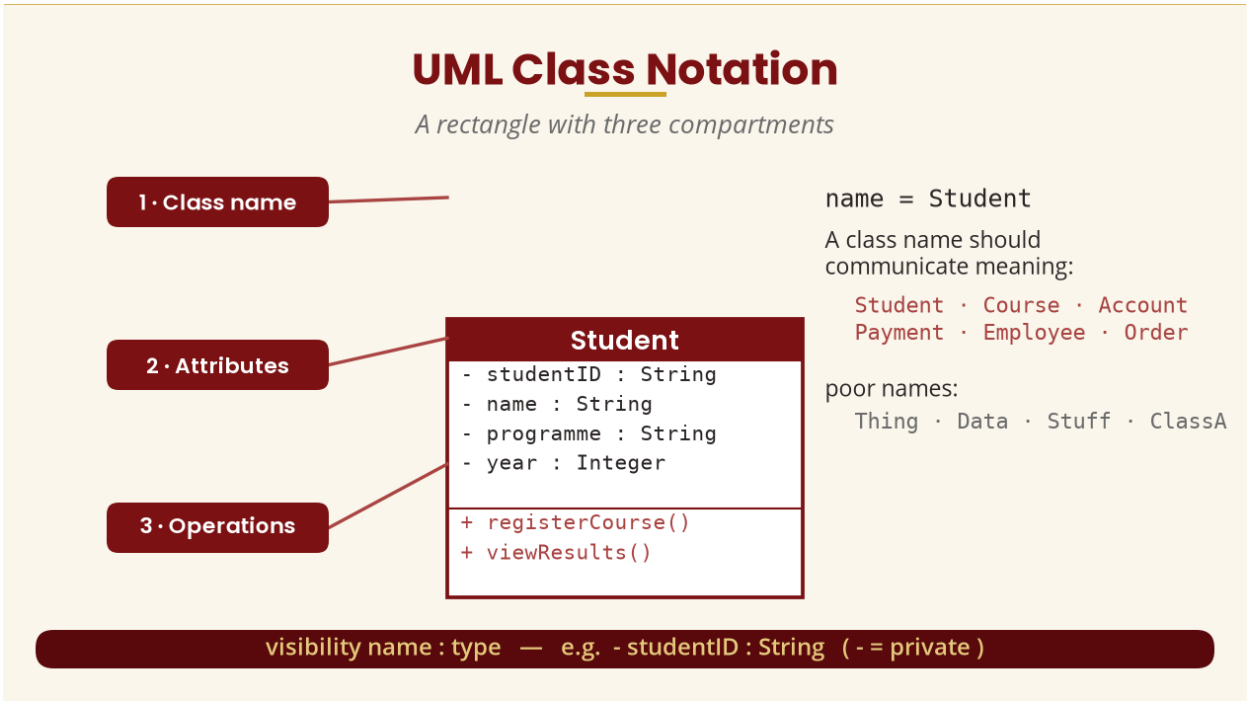

Figure 3 — UML class notation: the three compartments

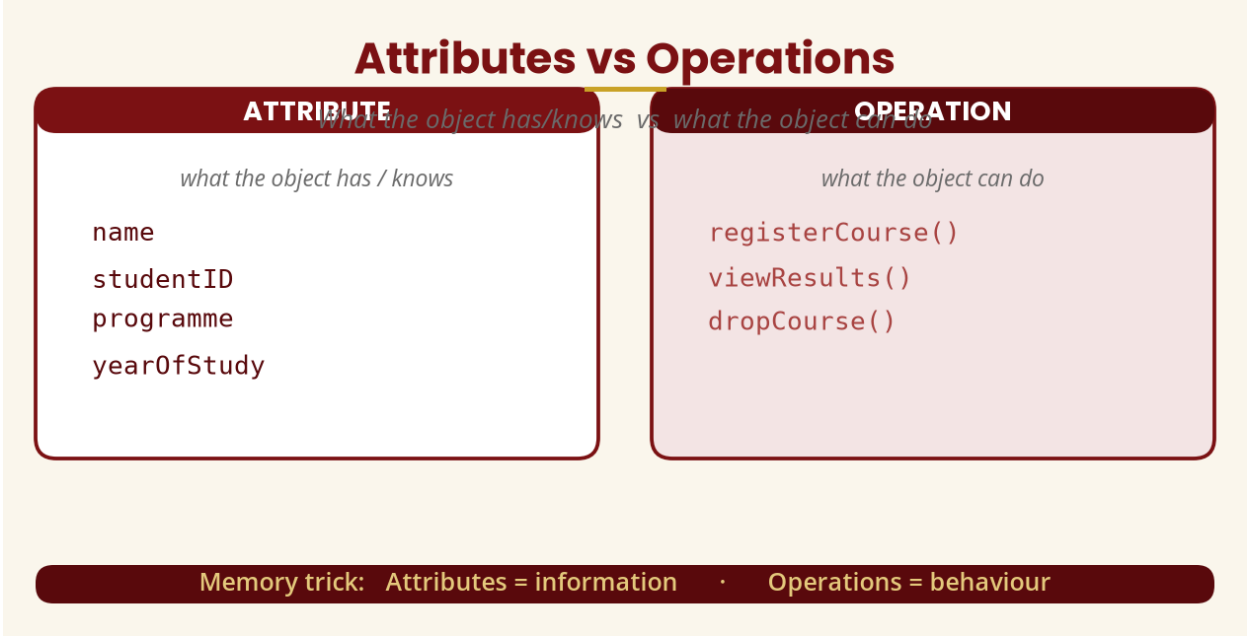

Figure 4 — Attributes vs operations

7. Object Notation

An object is commonly shown as a rectangle with its **name underlined**, followed by its attribute values. The key distinction: a class describes possible structure, while an object shows a specific instance and its values.

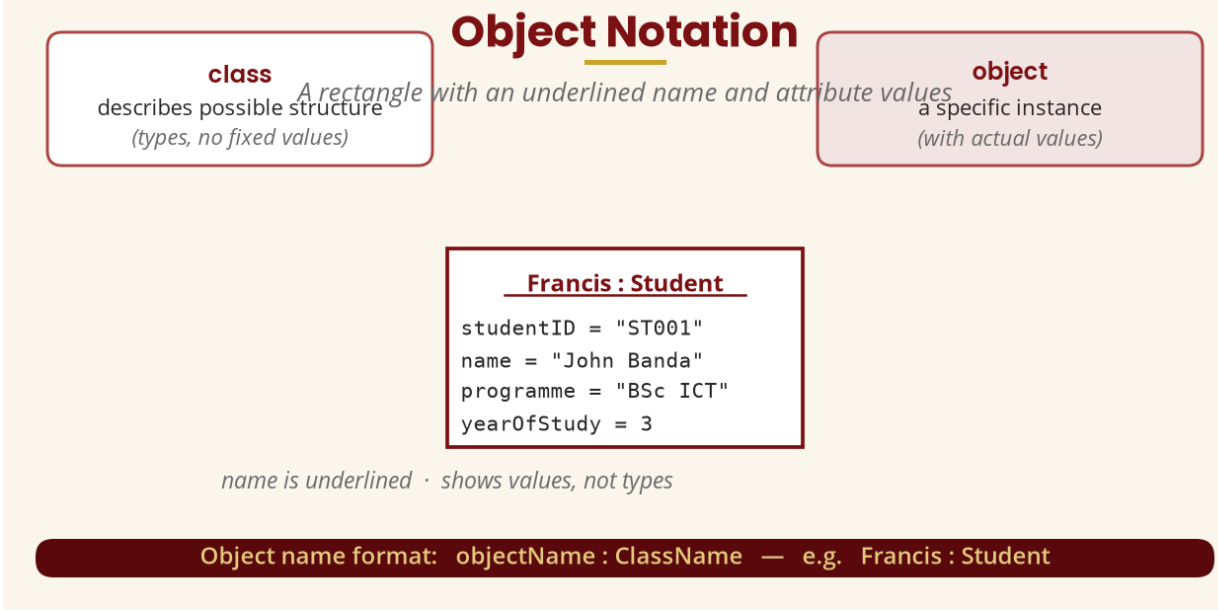

Figure 5 — Object notation: underlined name and attribute values

8. The Object Modelling Process

A simple, beginner-friendly process is: Problem Description Identify Candidate Objects Identify Candidate Classes Identify Attributes Identify Responsibilities/Operations Identify Relationships Validate the Model Draw the UML Class Diagram. **Do not simply underline every noun and call it a class** — that is too simplistic.

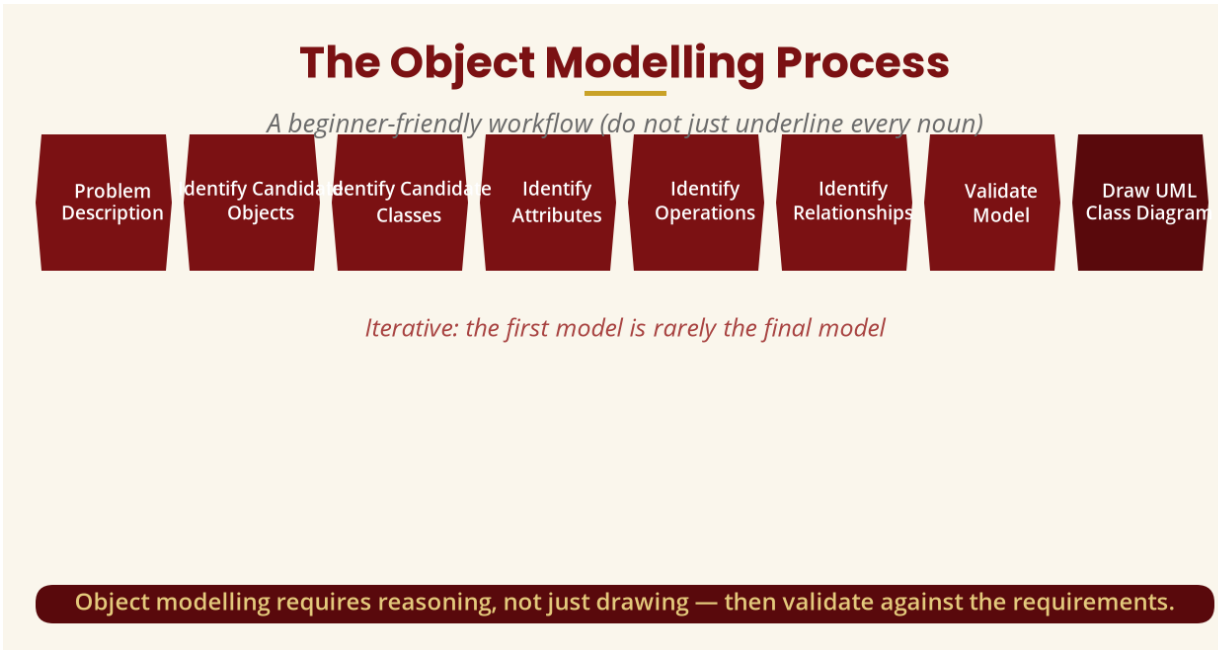

Figure 6 — The object modelling process (iterative)

9. Identifying Candidate Classes

Consider the sentence: *"A student registers for a course. The course is taught by a lecturer."* The potential nouns — student, course, lecturer — are strong candidate classes. Now consider: *"A student registers for a course on Monday."* The noun **Monday** should not automatically become a class — it is more likely an attribute/value associated with a date or registration.

The **noun technique** — look for important nouns in the problem description — is a useful starting point, but students must still ask: Is this concept relevant? Does the system need to store it? Does it have meaningful properties? Does it participate in relationships? Does it have behaviour/responsibility?

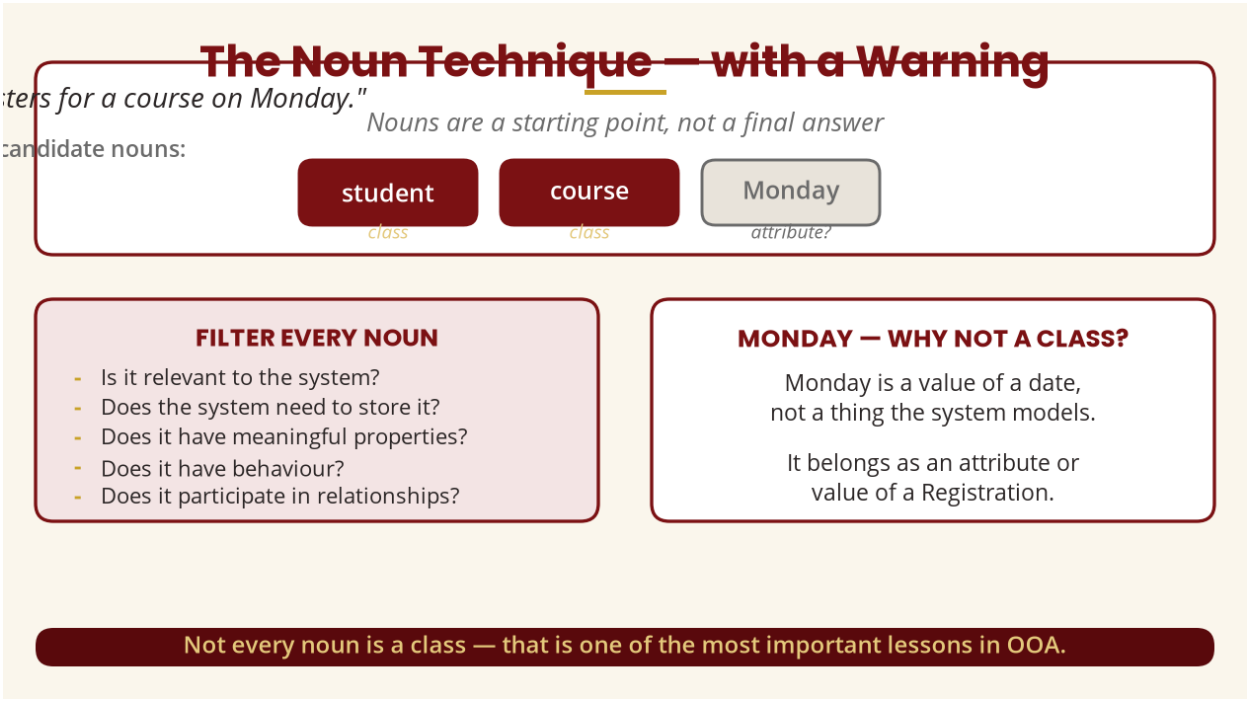

Figure 7 — The noun technique, with its warning

10. Links and Associations

A **link** represents a connection between particular objects — if Francis : Student is registered for BICT3202 : Course, we draw `Francis : Student ——— BICT3202 : Course`. An **association** describes a relationship between classes — `Student ——— Course` means instances of Student can be related to instances of Course.

The distinction is extremely important: **association connects classes; link connects objects.**

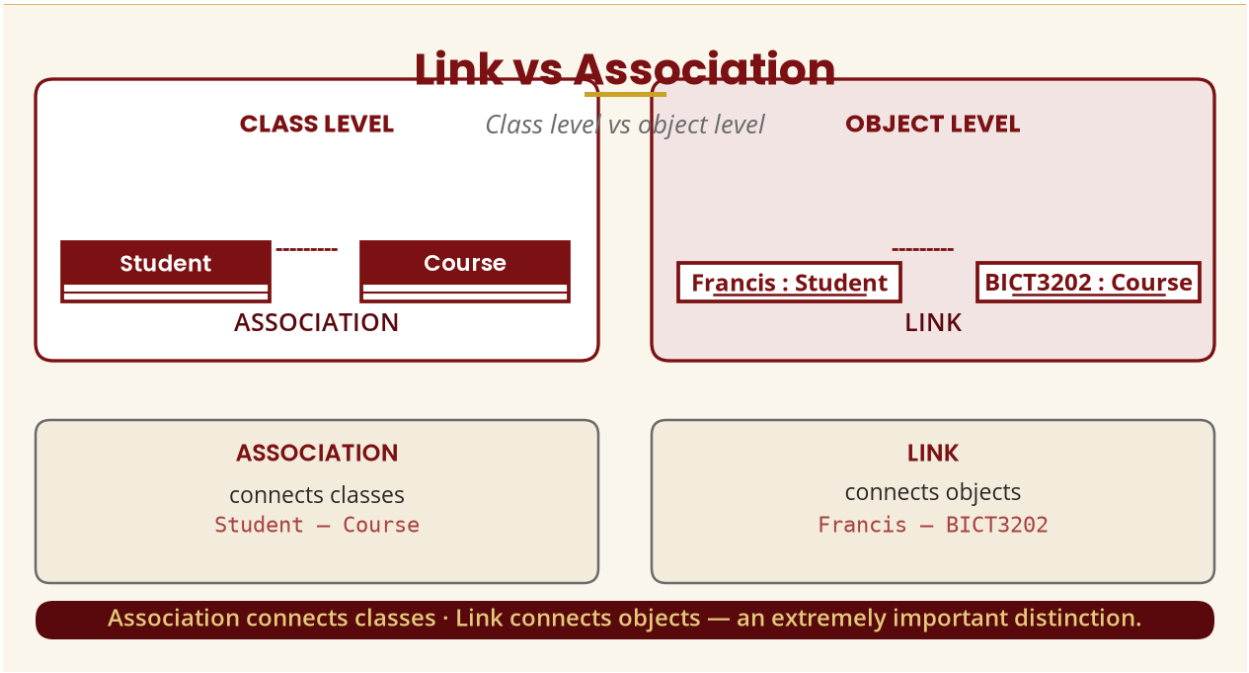

Figure 8 — Link vs association: class level vs object level

11. Naming an Association & Roles

An association can be given a meaningful name: instead of a bare line, write `Student ── registers for ── Course`, or `Lecturer ── teaches ── Course`, or `Customer ── places ── Order`. The name helps people understand the meaning of the relationship.

An association can also indicate the **role** each class plays — at the Course side the role could be *registeredCourse*; at the Student side, *student*. Roles are particularly useful when a relationship needs clarification.

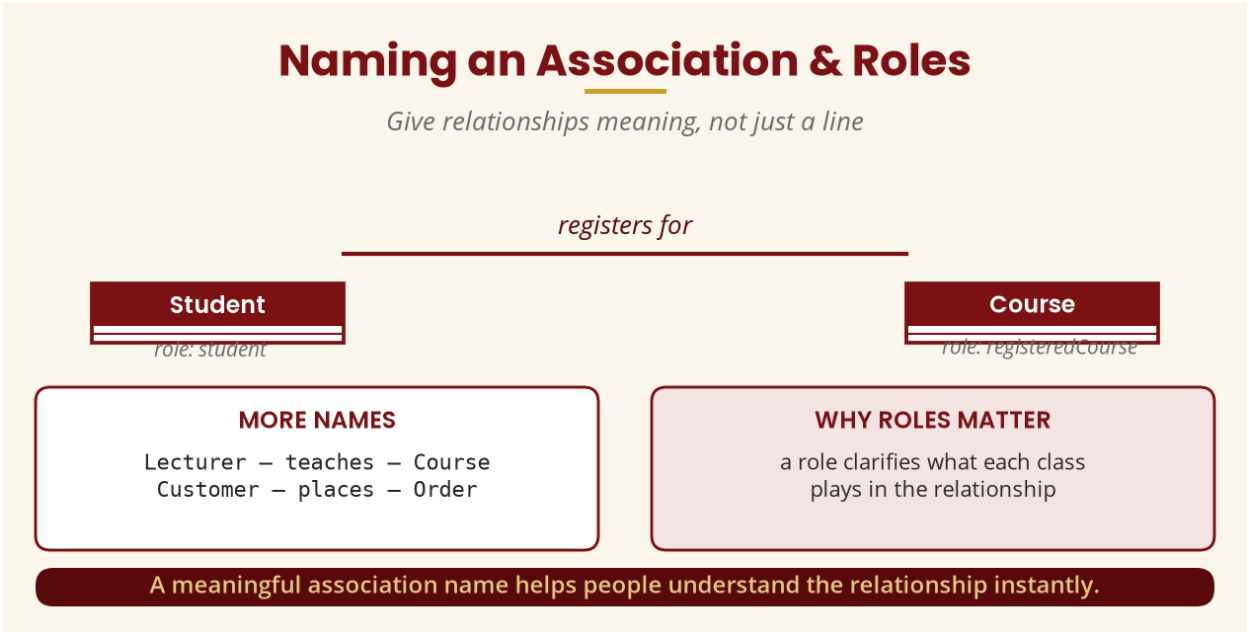

Figure 9 — Association names and roles

12. Multiplicity

Multiplicity tells us how many instances can participate in a relationship. `Student 1 ——— 0..* Course` means one Student can be associated with zero or many Courses. Why zero? Because a newly admitted student may not yet have registered for any course.

`Department 1 ——— 1..* Lecturer` means a Department must have at least one Lecturer and can have many. The difference is that **0..* allows zero**, while **1..* requires at least one**. Students should not memorise these without understanding them.

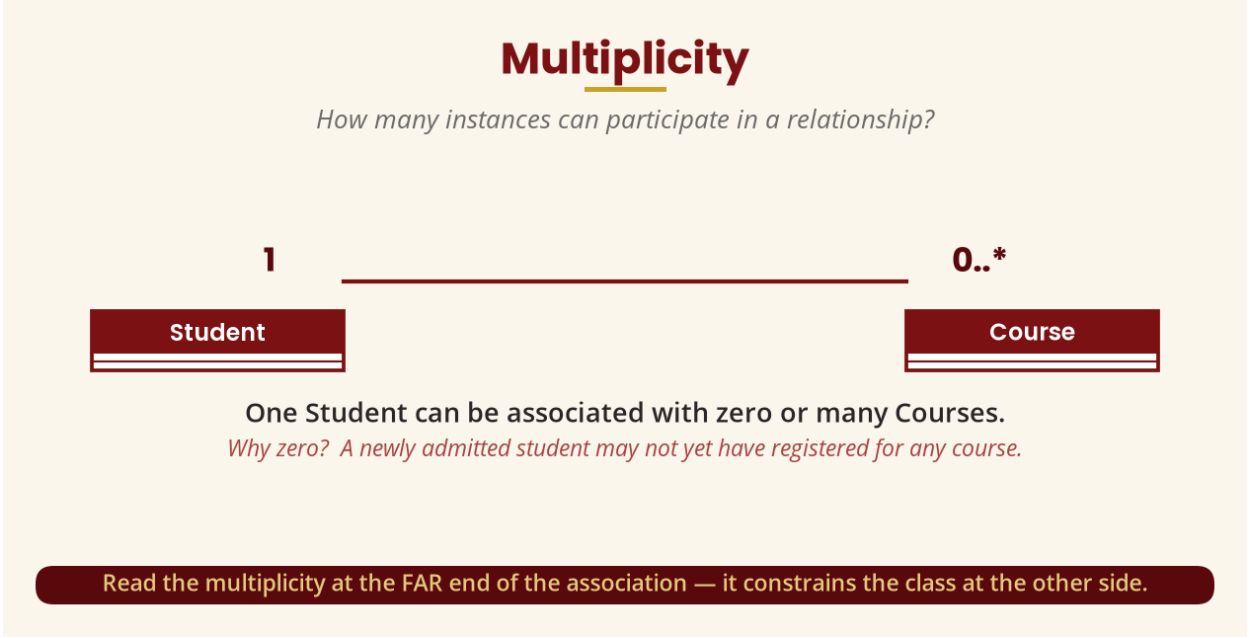

Figure 10 — Reading multiplicity at the far end

Notation	Meaning
1	Exactly one
0..1	Zero or one
*	Many
0..*	Zero or many
1..*	One or many
2..5	Between two and five

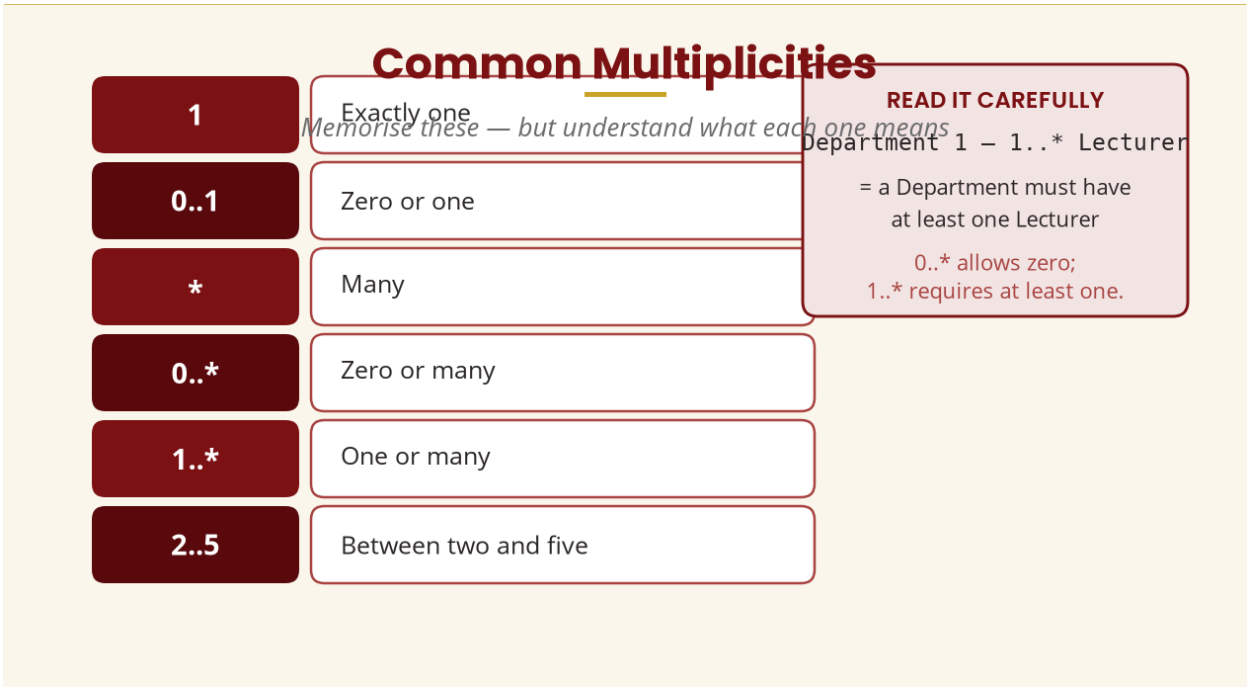

Figure 11 — Common multiplicities

13. One-to-One, One-to-Many, Many-to-Many

One-to-one: "Each student has one ID card, and each ID card belongs to one student" — `Student 1 ——— 1 IDCard`. **One-to-many:** "One lecturer teaches many courses" — `Lecturer 1 ——— 0..* Course`. **Many-to-many:** "A student can register for many courses, and a course can have many students" — `Student 0..* ——— 0..* Course`.

A many-to-many relationship often leads to an important modelling question: **does the relationship itself have information?** If registration has registrationDate, semester, status and grade, we may introduce a **Registration** class — this is revisited in greater depth later in the module.

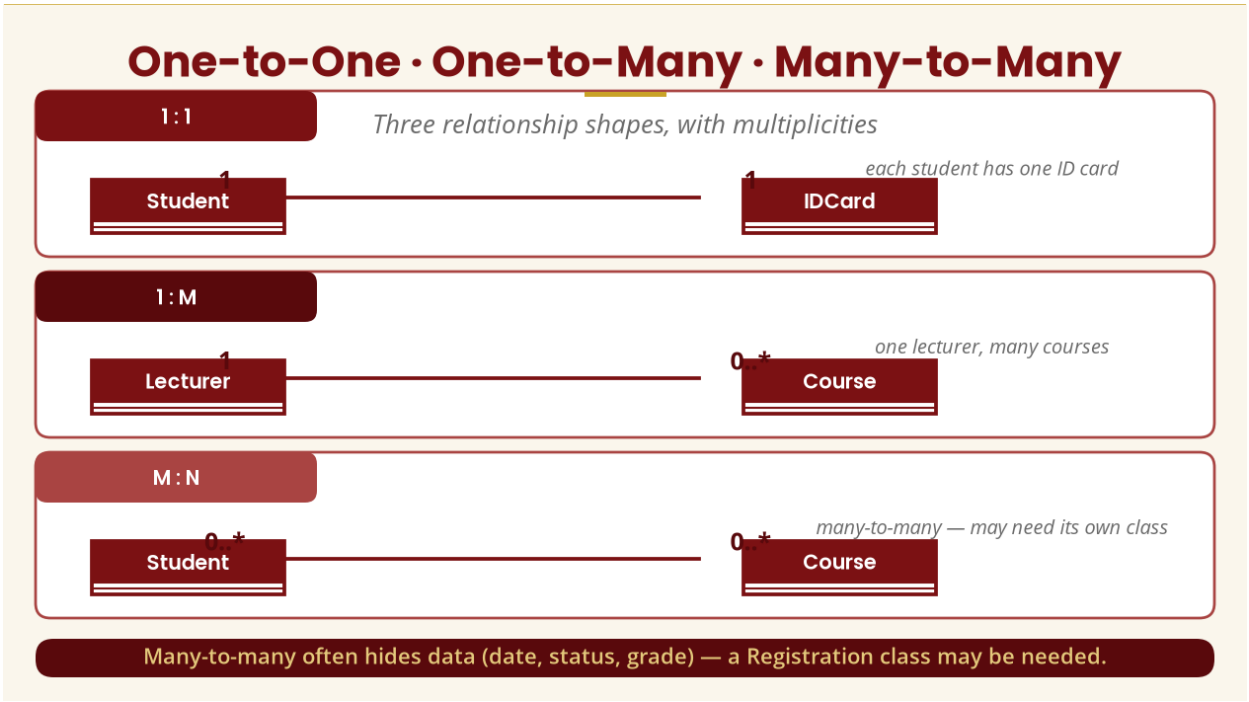

Figure 12 — The three relationship shapes

14. Generalization

Generalization represents a relationship between a more general class and a more specific class. For example, **User** is the general class, while **Student** and **Lecturer** are specialised classes. We read `Student` ——▷ `User` as "A Student is a User" — often called an **"is-a"** relationship.

Placing common features (userID, name, email, login(), logout()) in **User** and letting Student and Lecturer specialise avoids duplicating those features in every subclass — this supports reuse and reduces duplication.

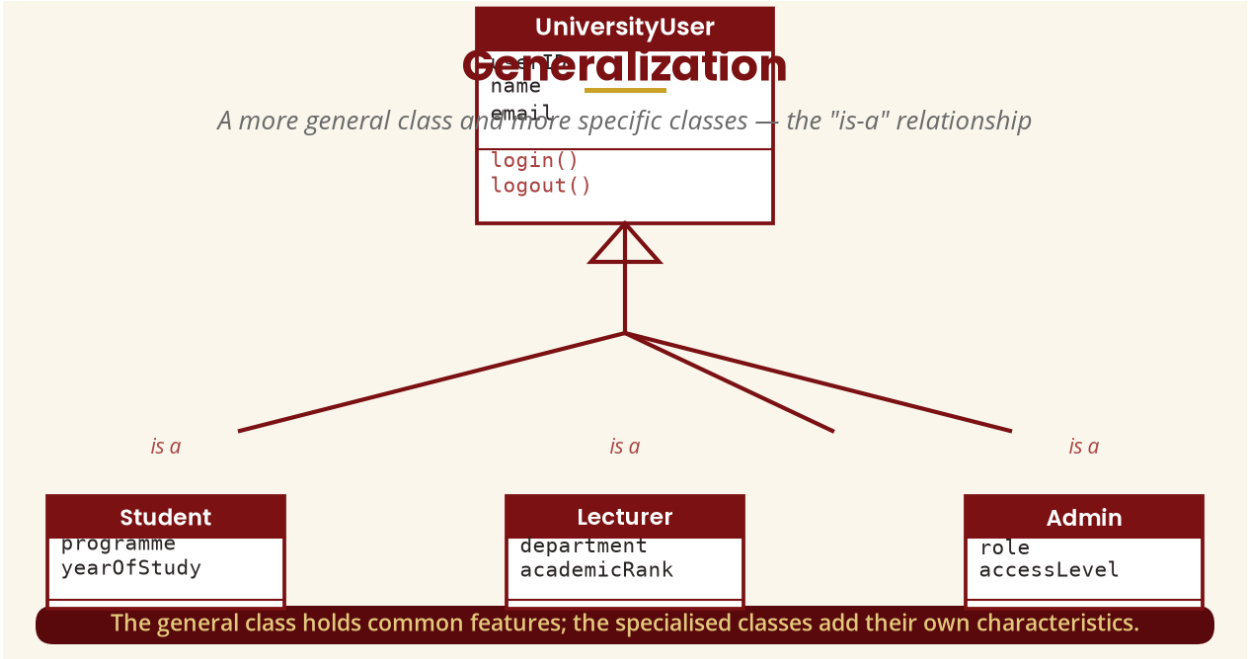

Figure 13 — Generalization: a general class and its specializations

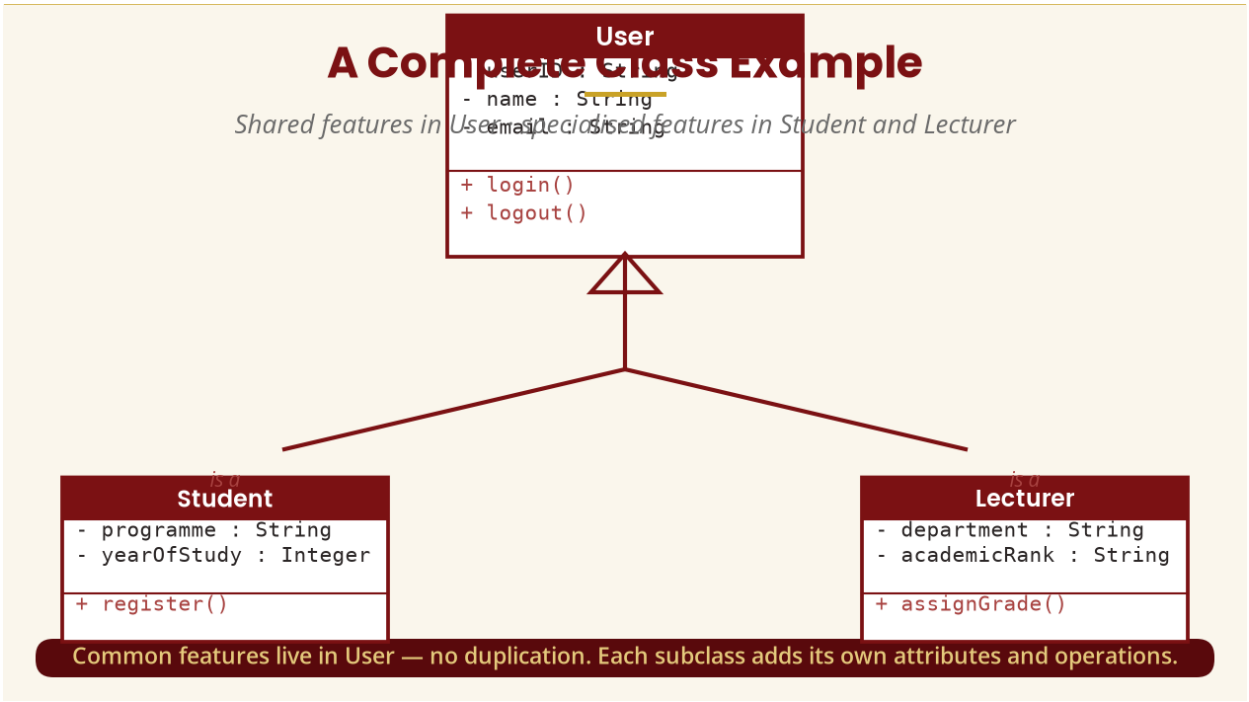

Figure 14 — A complete class example with shared features

15. Generalization vs Association

This distinction is extremely important. Generalization (`Student` ——▷ `User`) means **Student is a User**. Association (`Student` ——— `Course`) means **Student is related to Course**. Consider vehicles: a Car is a Vehicle (generalization), but **Car □□□□□ Driver** is not generalization — a car is not a driver; it is a relationship between two different concepts.

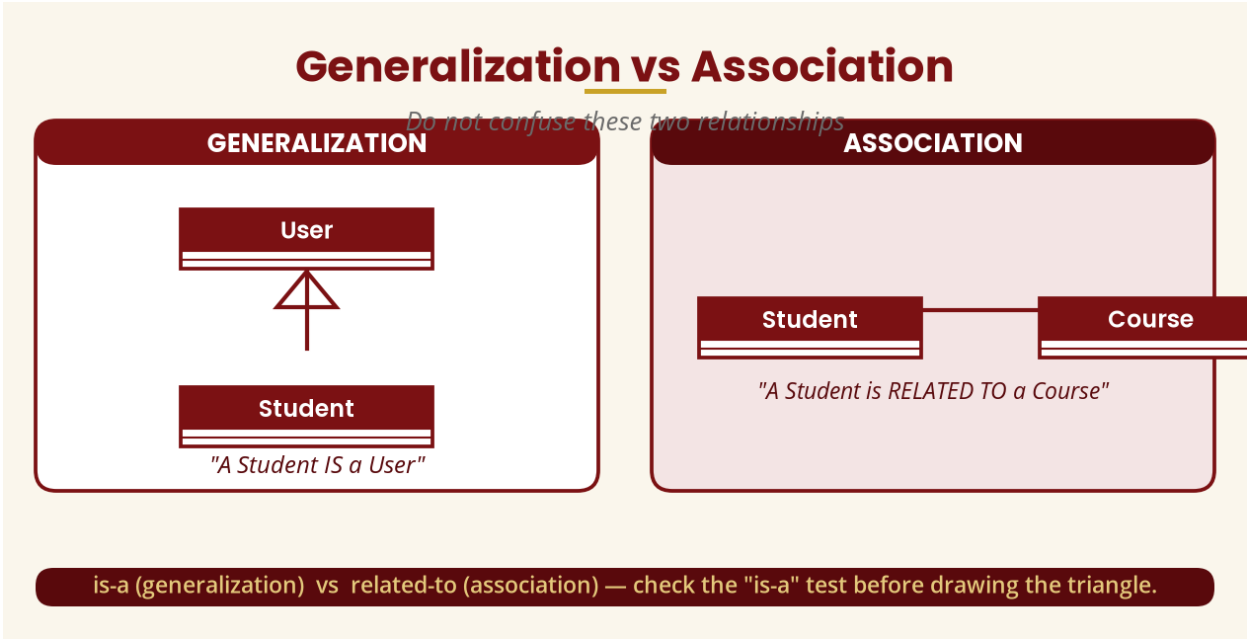

Figure 15 — Generalization vs association

16. Specialization, Vocabulary and the Tests

Specialization is the process of creating more specific concepts from a general concept — from Employee we may specialise Lecturer, Administrator and Technician. Generalization and specialization are **opposite perspectives** on the same structure: from the bottom upward, "Car is generalized into Vehicle"; from the top downward, "Vehicle is specialized into Car."

Vocabulary: the general class is also called the **parent**, **superclass** or **generalized class**; the specialised class is also called the **child**, **subclass** or **derived class**.

Two useful tests: the **"is-a" test** — "Is a Student a User?" Yes generalization may be appropriate; "Is a Student a Course?" No use an association. The **"has-a" test** — "Car has an Engine" points toward aggregation/composition (Week 6), which is different from "Car is a Vehicle" (generalization).

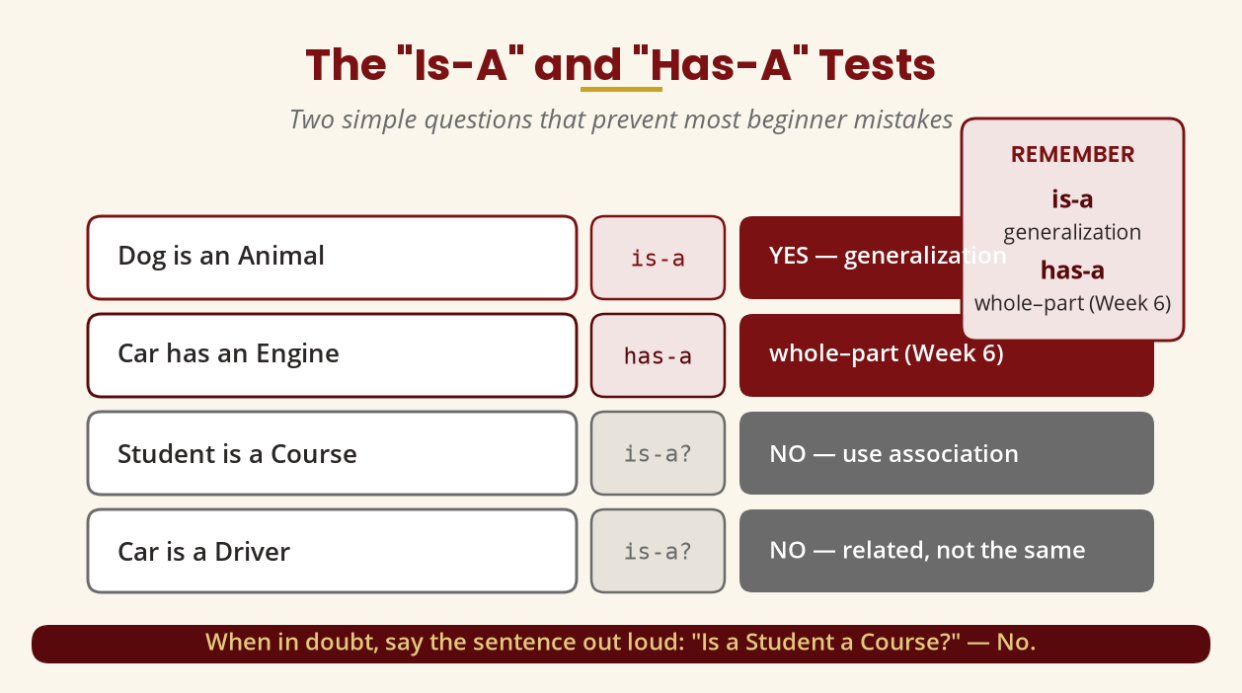

Figure 16 — The is-a and has-a tests

17. CASE Tools

The course outline requires a CASE tool demonstration (Select Enterprise). **CASE** means **Computer-Aided Software Engineering**. A CASE tool can support modelling, diagram creation, documentation, requirements management, design and code-related workflows. Common UML/modelling tools include Enterprise Architect, Visual Paradigm, StarUML, PlantUML, diagrams.net/draw.io and Modelio.

The important point is not the brand of the tool. **A CASE tool assists modelling; it does not replace analysis.** The tool can create a Student box, but it does not know whether Student should actually be a class — that decision comes from requirements, domain knowledge, analysis and business rules. **The analyst makes modelling decisions; the CASE tool helps represent and manage those decisions.**

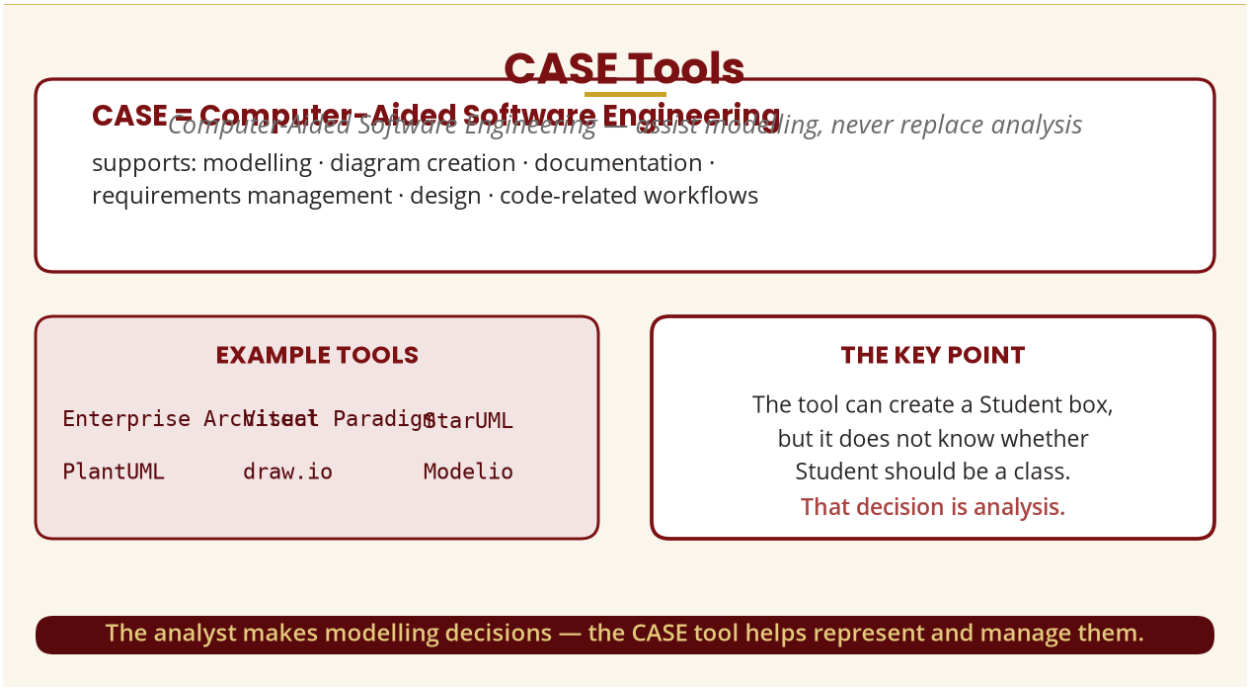

Figure 17 — CASE tools assist modelling, never replace analysis

18. Practical Lab — Create Your First Class Diagram

Using the selected UML CASE tool, create four classes — Student, Course, Lecturer, Registration — with the following attributes and operations, then add associations and multiplicities.

Class	Attributes	Operations
Student	studentID : String · name : String · programme : String · yearOfStudy : Integer	registerCourse() · dropCourse() · viewResults()
Course	courseCode : String · courseName : String · creditHours : Integer	addStudent() · removeStudent()
Lecturer	lecturerID : String · name : String · department : String	assignGrade() · viewClassList()
Registration	registrationID : String · registrationDate : Date · status : String	confirm() · cancel()

- Add associations: Student □□□ makes □□□ Registration · Registration □□□ is for □□□ Course · Lecturer □□□ teaches □□□ Course.
- Add multiplicities (document your assumptions): Student 1 □□ 0..* Registration · Course 1 □□ 0..* Registration · Lecturer 1 □□ 0..* Course.
- Add generalization: User □ Student and User □ Lecturer, then discuss which attributes should move up to User.

19. Validating a Class Diagram

After creating a diagram, do not submit it immediately — perform a model review. Ask:

- Does every class have a purpose? If not, remove or reconsider it.
- Does every attribute belong to the correct class? (studentName probably belongs to Student.)
- Are operations meaningful? Don't add random methods simply to fill the box.
- Are relationships logically correct?
- Are multiplicities supported by requirements?
- Are generalizations genuine "is-a" relationships?
- Is the diagram readable?

20. Common Student Mistakes

Mistake 1 — Confusing class and object.

"John is the Student class" is wrong thinking. Student = class; John : Student = object.

Mistake 2 — Using generalization everywhere.

Student □□□□□□ Course is wrong — a Student is not a Course.

Mistake 3 — Forgetting multiplicities.

Drawing Student □□□□□ Course and stopping; requirements often require us to know "how many?"

Mistake 4 — Making every noun a class.

Not every noun is a class.

Mistake 5 — Adding operations randomly.

eat(), sleep(), walk() are not system-relevant responsibilities — OOAD is about what the system needs, not everything a human can do.

Analysis vs implementation: at this stage, do not become obsessed with programming syntax. `public void registerCourse()` is implementation-oriented; at the analysis level we simply model `registerCourse()`. Understand the system first; implementation details grow in importance during design.

21. Object Modelling Exercise — Hospital

Scenario: *"A hospital has doctors, nurses and patients. A patient can have multiple appointments. Each appointment is with a doctor. A doctor belongs to a department. A patient has a patient number, name and date of birth."*

- **Task 1 — Candidate classes:** Patient, Doctor, Nurse, Appointment, Department.
- **Task 2 — Attributes:** Patient — patientNumber, name, dateOfBirth.
- **Task 3 — Relationships:** Patient □□□ has □□□ Appointment · Appointment □□□ with □□□ Doctor · Doctor □□□ belongs to □□□ Department.

- **Task 4 — Multiplicity:** Patient 1 □□□□□ 0..* Appointment.

WHY OBJECT IDENTIFICATION IS DIFFICULT

A real system does not come with labels saying THIS IS A CLASS, THIS IS AN ATTRIBUTE, THIS IS AN OPERATION. The analyst has to determine these. "The customer uses a shopping cart" — should ShoppingCart be a class? Usually yes, if it has meaningful state (items, total) and behaviour (addItem(), removeItem(), calculateTotal()). **Object modelling requires reasoning, not just drawing.** It is also iterative: the first model is rarely the final model.

22. Week 5 Summary and Key Terms

Term	Simple meaning
Object	A particular instance
Class	Description of a type of object
Attribute	Property/information held by an object
Operation	Behaviour/responsibility
Link	Connection between object instances
Association	Relationship between classes
Multiplicity	How many instances can participate
Generalization	General-to-specific relationship
Specialization	Creating a more specific class
Superclass / Subclass	General/parent class · specialized/child class
CASE	Computer-Aided Software Engineering
Object model	Representation of objects/classes and their relationships

THE BIG PICTURE

The progression is: real-world problem □ read requirements □ identify important concepts □ candidate objects/classes □ attributes □ responsibilities/operations □ links/associations □ multiplicities □ genuine generalization □ UML class diagram □ validate against requirements. **Object-Oriented Analysis is not primarily about drawing boxes — it is about understanding a problem domain and deciding what objects/classes, properties, behaviours and relationships are necessary to represent it.** Week 6 extends this model with aggregation, inheritance, polymorphism and grouping.

23. Examination-Style Question

Scenario: A university has students and lecturers. Each student has a student ID, name, programme and year of study. Each lecturer has a staff ID, name and department. Students register for courses. Each course has a course code, title and credit hours. Lecturers teach courses. A student may register for many courses, while each course may have many students.

Required: (a) identify the candidate classes [5]; (b) identify appropriate attributes [10]; (c) identify operations that could reasonably belong to the classes [5]; (d) draw a UML class diagram showing the associations [10]; (e) add appropriate multiplicities [5]; (f) identify whether generalization is appropriate and justify your answer [5]. **Total: 40 marks.**

24. References and Further Reading

Primary authoritative reference

- Object Management Group. (2017). *Unified Modeling Language (UML), Version 2.5.1.* OMG — the formal UML specification and the most authoritative source for notation, semantics, classes, objects, relationships and generalization.
- Object Management Group. *UML* — the UML specification together with machine-readable artefacts, including the UML abstract syntax metamodel.

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*